

Deep Reinforcement Learning: Bipedal Walker & Cart Pole with DQN

Introduction to Deep Learning — Project 21

Lea Seifried Michael Haggmann Dominik Rzecki

Technical University of Vienna

March 24, 2026

Outline

- 1 Introduction and Overview
- 2 Theoretical Background
- 3 Implementation
- 4 Experiment Setup
- 5 Results and Discussion

Introduction and Overview

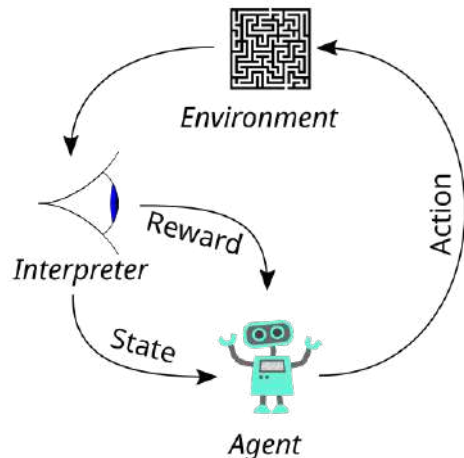
Problems and Agents

Train agent, such that it solves a specific Problem ($\Leftrightarrow$ performs Task).

Agents Problem: Perform that task correctly.

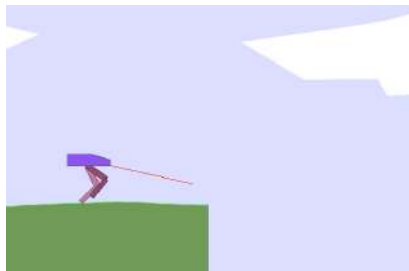
Engineers Problem: Train such agent with least amount of effort.

$\Rightarrow$ RL



Our Main Problem: Bipedal Walker from Gymnasium

Make stickman walk.



Environment:

- Flat slightly uneven slab, Newtonian Physics

Agent:

- lives in 2D
- 4 controllable joints (speed)

⇒ 6 total d.o.f., 4 controllable

Observation: Hull angle, angular velocity, linear velocities (x , y), Joint angles & angular velocities, Binary ground-contact info 10 LIDAR range measurements
⇒ **24-dimensions**

Rewards, Penalties & No-Gos

Gymnasium provides interpreter.

Reward components:

- + Positive reward for **forward displacement**
- Large penalty (-100) for **falling**
- Small penalties for **motor usage** (energy cost)

Episode terminates when...

- 1 Torso contacts the ground (fall)
- 2 End of terrain is reached
- 3 Time limit exceeded

Objective

Maximize cumulative discounted reward — keep walking, don't fall.

(We created Wrapper with reward shaping, more on that later.)

Our Approach: Deep Q-Networks

(tabular)-RL:

- Optimize sequential decision-making, under uncertainty through interaction and feedback.
- memorization, no compression (inf. obs. space $\Rightarrow$ inf. LUT)



DQN: (RL + DNN)

- Replace table with DNN to approximate Q^*
 - Use Q-Function. (no model / policy derivation needed)
- $\Rightarrow$ continuous observation space, generalization
- $\Rightarrow$ still discrete action space :(
- $\Rightarrow$ challenges: sample efficiency, stability

DNN/MLP:

- Universal function approximation
- Can compress / generalize.
 $\Rightarrow$ continuous observation space

Theoretical Background

Reinforcement Learning: Core Concepts

Markov Decision Process (S, A, P, R, γ) :

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots$$

- $s_i \in S$ State
- $a_i \in A$ Action
- $r_i \sim R(s_i, a_i)$ Reward
- $P(s'|s, a)$ Transition Probabilities

Discounted Return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad \gamma \in [0, 1)$$

Policy $\pi(a|s)$: **Learned** mapping from states to a distribution over actions.

Objective: Find a $\pi(a|s)$, such that $\mathbb{E}[G_t]$ is maximized.

DQN is **model-free** — no access to model dynamics $P(s'|s, a)$.

Q-Learning: Action-Value Function

Goal: Estimate the return G_t during a trajectory, i.e., learn the “goodness” of a state-action pair (s_t, a_t) .

Action-value function (policy-dependent):

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid s_t = s, a_t = a]$$

Optimal action-value function:

$$Q^*(s, a) = \max_{\pi} Q^\pi(s, a)$$

Goal: Learn an optimal action-value function $Q(s, a)$ from interaction data.

Training loop:

- 1 Interact with environment using policy e.g. ϵ -greedy
- 2 Store transition in replay buffer:

$$(s_t, a_t, r_{t+1}, s_{t+1}, \text{done})$$

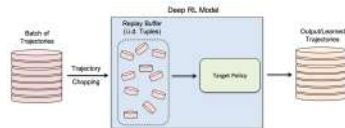
- 3 Sample random minibatch from replay buffer
- 4 Compute Bellman target y_t
- 5 Update network by minimizing TD error (DNN $\rightarrow$ gradient descent)

Replay Buffer: From RL to Supervised Learning

Goal: Approximate optimal $\pi^*(s) = \arg \max_a Q^*(s, a)$.

Replay buffer stores experience:

$$(s_t, a_t, r_{t+1}, s_{t+1}, \text{done})$$



Why replay buffer?

- Breaks temporal correlations in data (notice i.i.d assumption)
- Reuse of past experience
- Improves sample efficiency

RL $\rightarrow$ supervised learning by creating targets from experience.

DQN Learning: Bellman Target & Loss

Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \right]$$

Bellman target (DQN):

$$y_t = \begin{cases} r_{t+1}, & \text{if done} \\ r_{t+1} + \gamma \max_{a'} Q_{\theta-}(s_{t+1}, a'), & \text{otherwise} \end{cases}$$

Bootstrap principle: Estimate future return using current Q-function.

Loss (TD error / MSE):

$$L(\theta) = \mathbb{E} \left[(y_t - Q_{\theta}(s_t, a_t))^2 \right]$$

Terminal handling: No future reward is added when done = true.

Greedy Policy

Always pick $a = \arg \max_{a'} Q(s, a')$.

$\Rightarrow$ No exploration; gets stuck.

ϵ -Greedy

$$a = \begin{cases} \text{random} & \text{with prob. } \epsilon \\ \arg \max_a Q(s, a) & \text{otherwise} \end{cases}$$

ϵ is annealed over training (explore $\rightarrow$ exploit).

Boltzmann (Softmax) Exploration

$$\pi(a|s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{a'} \exp(Q(s, a')/\tau)}$$

Temperature τ is annealed over training.

Actions with higher Q-values are *proportionally* more likely — softer than ϵ -greedy.

Target Network: Solving Moving Target Problem

Problem: Using the *same* network for both prediction and target:

$$\text{Target} = r + \gamma \max_{a'} Q(s', a'; \theta)$$

Every update shifts the target $\Rightarrow$ feedback loop $\Rightarrow$ oscillation / divergence.

Solution — Target Network θ^- :

- Identical copy of online network, weights *frozen*
- Used *exclusively* to compute TD target
- Synced to θ every C steps: $\theta^- \leftarrow \theta$

DQN Target

$$y^{\text{DQN}} = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

Effect

RL temporarily becomes a *supervised* task with a stable objective — decouples weight updates from target calculation.

Double DQN Learner

online network Q_θ + *target* network $Q_{\theta-}$.

Double DQN target (decouples selection from evaluation):

$$y_t = r_t + \gamma Q_{\theta-} \left(s_{t+1}, \underbrace{\arg \max_{a'} Q_\theta(s_{t+1}, a')}_{\text{online selects action}} \right) (1 - d_t)$$

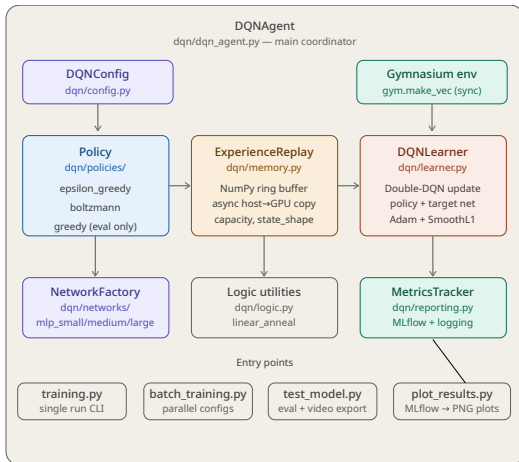
- Target sync every `target_sync_interval` epochs
- Hard copy ($\tau = 1$) by default; optional Polyak averaging

Why Double DQN?

Standard DQN tends to *overestimate* Q-values because the max-operator uses the same network for selection and evaluation. DDQN reduces this bias.

Implementation

System Architecture



Key Facts

- Action Space Discretization (2 bins)
- MLP
- Experience Replay Buffer
- Target Network Update
- greedy, ϵ -greedy & Boltzmann
- Double-DQN

Multilayer Perceptron (MLP) approximating $Q^*(s, a)$:

Key	Hidden layers	Use
mlp_small	128 $\rightarrow$ 128	Ablation baseline
mlp_medium	256 $\rightarrow$ 128 $\rightarrow$ 64	Ablation
mlp_large	512 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 64	Best results

- **Input:** 24-dimensional observation vector
- **Activation:** ReLU after each hidden layer
- **Output:** Q-value per discrete action (one neuron per action index)
- Registered in `NetworkFactory` for easy config-driven selection

Training Loop

Each epoch runs 3 phases:

- ① **Collection** — current policy interacts with environment(s) for `steps_per_epoch` steps; transitions stored in buffer.
- ② **Optimization** — `updates_per_epoch` mini-batches of size `batch_size` sampled → gradient steps.
- ③ **Target sync** — target network updated every `target_sync_interval` epochs.

Warmup — Additionally, the first epoch is preceded by Warmup: Fill replay buffer with randomly sampled experiences, without updating gradients.

Experiment Setup

Metrics reported:

- Mean & std of cumulative return
- Median, min, max, best
- Mean episode length (survival proxy)

Hyperparameters:

- Network $\in$ mlp- $\{\text{large, medium, small}\}$
- Exploration Policy
- Learning Rate
- Discount Factor (γ)
- Replay Buffer Size
- Batch Size
- Warmup Steps
- Target Sync Interval
- Exploration (ϵ)
- ...

Ablation Study Overview

Multiple configurations were trained varying one or more hyperparameters at a time:

Cat.	Variable	What we tested
01	Exploration rate	Baseline vs. higher ϵ_{end} (0.05 $\rightarrow$ 0.10)
02	Learning rate	3×10^{-4} vs. 10^{-4}
03	Architecture	mlp_medium vs. mlp_large
04	Replay + Batch	Larger buffer & batch size combo
05	Discount γ	0.99 vs. 0.97
06	Exploration type	Boltzmann vs. ϵ -greedy

Results and Discussion

BipedalWalker: Top 5 runs

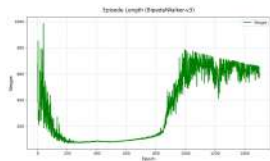
R	Buffer	Start	End	LR	Policy	Mean	Std	Med.	Best
1	500k	2.0	0.2	$5 \cdot 10^{-5}$	Boltzmann	329.16	49.99	321.32	513.25
2	500k	1.0	0.1	$5 \cdot 10^{-5}$	ϵ -greedy	243.00	50.80	229.60	413.00
3	200k	1.0	0.1	$2.5 \cdot 10^{-4}$	ϵ -greedy	167.95	9.42	167.20	188.44
4	100k	1.0	0.1	$3 \cdot 10^{-4}$	ϵ -greedy	163.71	12.57	164.61	189.30
5	200k	2.5	0.1	$2.5 \cdot 10^{-4}$	Boltzmann	160.57	14.96	159.09	195.71

At similar settings there are no big differences between runs.

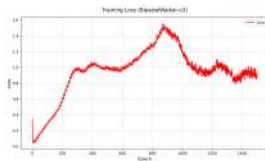
- **Boltzmann Exploration:** Best results, but required extensive tuning.
 - Proportional exploration: better actions explored more often
 - Avoids purely random, wasteful moves
 - Richer learning signal
- ϵ -Greedy achieved the lowest standard deviation.
 - Stable and predictable
 - Lower performance ceiling
- **MLP capacity** most important factor
 - `mlp_large` $\gg$ `mlp_medium`
- Discount factor γ strongly influences long-term planning.
- Learning rate is highly sensitive and requires tuning.
- Long-horizon discount ($\gamma = 0.99$) is beneficial.
- Large network and large replay buffer good

Successful Parameters

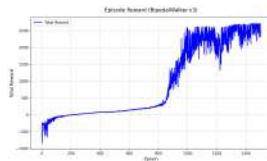
Observation: Long plateau and abrupt rise.



Ep. Length



Ep. Loss



Ep. Reward

Key dynamics:

- Loss increases initially, then decreases as reward shoots up.
- Episode length increases alongside reward

Best Model:

- Solved Episode Length: $329.16 \geq 300$
- mlp_large + Boltzman

Component

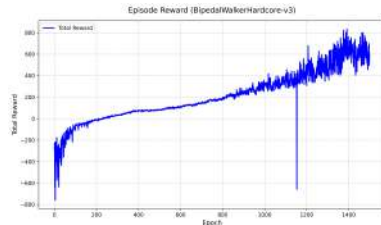
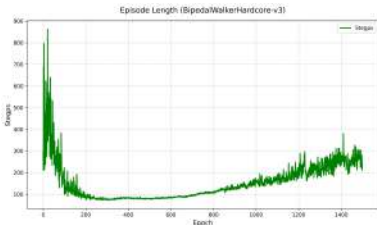
Value

Network / Policy	mlp_large + Boltzmann
Learning rate	5×10^{-5}
Discount factor (γ)	0.99
Batch size	256
Replay buffer	500k
Warmup	50k steps
Training setup	1500 epochs, 1000 stp./ep.
Exploration	$2.0 \rightarrow 0.2$
Target sync	every 10 updates

Comparative: BipedalWalker vs. BipedalWalkerHardcore

How suitable is DQN for **Hardcore Variant**?

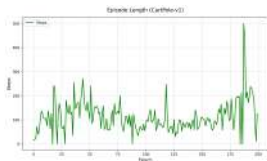
- Very uneven terrain.
- obstacles, stairs, pits
- Higher exploration requirements
- Obstacles → agent must learn more “when then” scenarios.
- World model would be good.
- Continuous action space would be good.



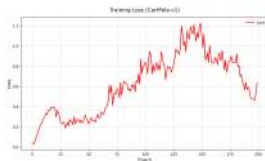
Same params as in “Default” variant (see above), do not bring episode length up well, agent fails → better hyperparams necessary.

Bonus: Successful CartPole Parameters

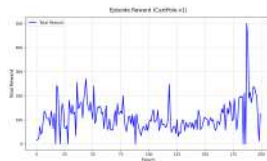
For comparative we also tested a different environment, CartPole-v1.



Ep. Length



Ep. Loss



Ep. Reward

CartPole-v1

- State: 4D — Action: 2D
- Task: Inverse pendulum on cart stabilization
- Action Space: already Discrete by default

Best Model:

- Solved Episode Length: $500 \geq 500$
- mlp_small + ϵ -greedy

Component

Value

Network / Policy	mlp_small + ϵ -greedy
Learning rate	1×10^{-3}
Discount factor (γ)	0.99
Batch size	64
Replay buffer	10k
Warmup	1000 steps
Training setup	200 epochs, 1k stp/ep., 200 upd./ep.
Exploration	0.5 $\rightarrow$ 0.05
Target sync	every update

Comparative: CartPole vs. BipedalWalker

Are params for high complexity good in low complexity?

CartPole-v1

- Simple Stabilization

BipedalWalker-v3

- Task: Complex locomotion

	CartPole	Bipedal	
Learning rate	10^{-3}	$5 \cdot 10^{-5}$	BW Rougher Q-landscape
Exploration	ϵ -greedy	Boltzmann	CP Needs stable Q-values
Network	mlp_small	mlp_large	BW must approx. more complex function
Replay Buffer	10k	500k	Larger state diversity

- Training CartPole with (best) Bipedal params achieves Return of 500 but training unnecessary long & expensive.
- Also Intuitively: $\exists$ simple classical control with PID $\Rightarrow$ Simple DQN should exist.

Roles

Lea Seifried: Training

Michael Hagmann: Model

Dominik Rzecki: Documentation, Slides

GenAI usage notice

- drafting of code & of documentation
- explanations & spec knowledge

The FASTEST introduction to Reinforcement Learning on the internet

Hasselt, Guez, Silver, “Deep Reinforcement Learning with Double Q-Learning”, Dec. 2015

Thank You!

Questions?

Lea Seifried:	<code>e12428595@student.tuwien.ac.at</code>
Michael Hagmann:	<code>e12426877@student.tuwien.ac.at</code>
Dominik Rzecki:	<code>e12410100@student.tuwien.ac.at</code>

Shaped Reward:

$$r = r_{\text{base}} - 2|\theta| - 0.5|\omega| - 0.5|v_y| + \begin{cases} 5v_x & v_x > 0 \\ -0.3 & v_x \leq 0 \end{cases}$$

Motivation:

- Fix sparse reward / local optima (raw 130 Δx fails)
- Stabilize posture (θ, ω, v_y penalties)
- Encourage forward motion, penalize stagnation

Gait Constraints:

- **Foot contact duration** $-0.1(\text{steps} - 15)$ per leg
- **High contact EMA** $-1.5(\text{EMA} - 0.75)$ per leg
- **Asymmetry penalty**
 $-(|\text{EMA}_1 - \text{EMA}_2| - 0.3)$
- **Hip inactivity** -0.3 if one hip inactive